

Differential Testing: How to find differences between programs that mostly behave identically?

Jonas Möller¹

Abstract: Differences between programs based on the same specification might lead to vulnerabilities that can not be detected by conventional testing. Differential testing is able to find these discrepancies by executing multiple programs on the same input and comparing their output. In this work, we discuss the fundamentals of differential testing and outline a general scheme for differential testing methods which is used to categorize and analyze current research approaches. Based on this, we formulate our own research questions which focus on how machine learning can aid differential testing.

Keywords: Differential Testing; Fuzzing

1 Introduction

An aspect that is often overlooked in software engineering is that although we expect systems of the same kind to behave identically they might exhibit minor discrepancies. Even when components adhere to the same specification, irregularities like underspecified functionality, misconceptions about API usage, or programming errors can lead to slightly different outputs for the same input. In the case of underspecified functionality, the discrepancies between programs are not defects per se, since each one implements the specification correctly; thus, they can not be caught by conventional test suites or even verification efforts. If these components are used in conjunction, their interplay might introduce vulnerabilities stemming from a different handling of the same data.

An example for such a vulnerability can be crafted using HTTP parameter pollution [Ba11]: If an HTTP request contains multiple values for the same parameter, a web application firewall and a server behind this firewall might process different values for the same parameter; this can be used to pass a benign value to the firewall and a malicious one to the server. Another example is the evasion of malware detectors. These need to process incoming files to inspect them (e.g. extract a zip archive) and thus have to recreate parts of the target application logic. Even for a single file type the detector has to recreate the functionality of multiple competing implementations which result in abusable discrepancies between detector and user application [JS12]. Other examples include file type inference [BCS09], SSL/TLS certificate verification [Br14; CS15] and HTTP request smuggling [Ja21]. Thus, finding differences between programs is not only relevant for interoperability,

¹ TU Braunschweig, Institut für Systemsicherheit, Mühlenpfordtstraße 23, 38106 Braunschweig, Deutschland, jo.moeller@tu-braunschweig.de

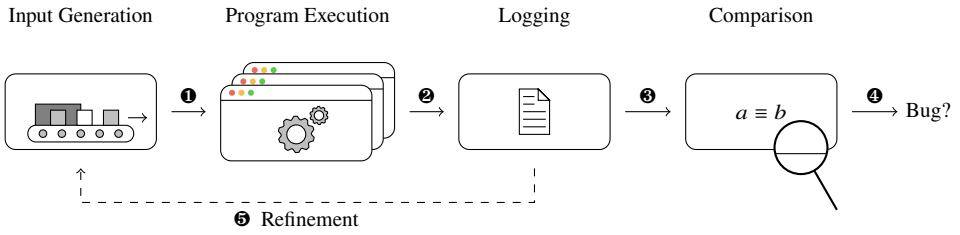


Fig. 1: The differential testing pipeline: Input is generated (①), passed to multiple programs (②), run time information is collected (③) and the output is checked for equivalence (④). If the input generation is guided, the run time information is used for refinement (⑤).

but is also a security-critical task. This is a hard task, since, in essence, we want to verify the equivalence of two programs, which is an undecidable problem.

One approach to systematically check programs for their equivalence is *differential testing*. By feeding the same inputs to multiple programs, we can use them as cross-referencing oracles, compare their outputs and detect subtle differences in their implementation. Although this does not guarantee program equivalence, it increases the trust in the correctness of programs—similar to conventional software testing.

2 Differential Testing

The basic idea of differential testing is pretty simple: If we feed the same input to multiple programs, the programs should return equivalent output. Differential testing approaches can usually be described by the pipeline shown in Figure 1: In each iteration, an input is generated and passed to the programs under test. Then, the outputs of the programs are logged and the results are compared to find differences. In the following, we describe the individual steps of the pipeline in more detail and—to give an overview of the current research literature—we discuss relevant aspects of differential testing papers for each step.

2.1 Input Generation

In the first step, the inputs for the programs are generated. Depending on the type of input expected by the program and the availability of source code, different input generation strategies can be used. In general, a good strategy creates inputs that execute unexplored program paths where uncommon edge-cases make it more likely to find differences. There are three aspects that characterize an input generation strategy: First, a strategy can be either guided or unguided. Guided methods use feedback from the program evaluations to refine the next inputs, whereas in unguided methods each iteration is independent of the previous iterations. For guided methods, the programs can be instrumented during compilation, if

the source code is available, to gather run time information like coverage measurements. Second, the inputs exhibit various degrees of structure: The simplest inputs are based on a random sequence of bytes while more sophisticated methods can employ a context-free grammar or modify an initial corpus. While unstructured, semi-random input can be easily generated, semantically sound input reaches deeper into program logic. Lastly, the input generation ranges from generative to transformative. The former creates inputs from a set of rules (e.g. a context-free grammar) while the latter applies transformations to a set of known inputs; hybrid approaches can combine both types. Since differential testing is based on fuzzing, the large body of research in this direction can be leveraged to improve discrepancy detection.

In the literature, Brubaker et al. [Br14] use an unguided, transformative approach by combining and randomly altering SSL/TLS certificates. This approach is improved upon by Chen; Su [CS15] who use coverage-guided Markov chain Monte Carlo (MCMC) methods which randomly select transformations. In the domain of testing Java Virtual Machine implementations, MCMC-based sampling of input transformations is also used by Chen et al. [Ch16]. For a generative approach, McKeeman [Mc98] discusses and implements seven different input tiers ranging from “Random sequence of ASCII characters” to “Model-conforming C programs” to differentially test C compilers at a greater depth. This work is improved upon in *CSmith* which adds further restrictions to dismiss all programs that include any undefined behaviour [Ya11]. Lastly, Holler et al. [HHZ12] and Jabiyeve et al. [Ja21] combine generative with transformative aspects for unguided fuzzing of language interpreters and HTTP servers respectively.

2.2 Program Execution

Executing the programs is largely an engineering problem: Each instance is given the input for the current iteration and is monitored to collect run time information as described in the logging phase. There exist two obstacles that need to be addressed when setting up a differential testing pipeline: First, passing input to a program is not always straightforward, so it is common to create a test harness, which acts as an adapter between the fuzzer and the fuzzing target and handles additional issues (e.g. if an application uses a GUI). Second, the functionality of programs may be influenced by external inputs like environment variables, configuration files or command line arguments. These either need to be semantically equivalent for each program under test or need to be treated as part of the input.

A rather sophisticated program execution stage is employed by Jabiyeve et al. [Ja21] who construct a controlled network environment of server-, proxy- and CDN-technologies. In this network, HTTP requests are used as inputs for the instances and HTTP messages between servers are monitored. This setup is used to find differences between server implementations which can be leveraged for HTTP request smuggling.

2.3 Logging

During its execution, a program is monitored and its results are logged for further analysis. The logging phase consists of two parts: First, the output of the programs is logged and used for the comparison phase. This is not trivial since the output of the programs should adequately reflect their functionality, but still needs to be comparable. Second, if the input generation uses a guided input generation approach, run time information for the refinement phase is collected. This can include coverage, standard output, program exit status or more detailed results from program analysis. Which analyses are feasible depends on the scope of the testing, e.g. if the source code of the program is available, the code can be instrumented to achieve more fine-grained run time information. Although more detailed analysis of the program leads to better refinement, it also comes at a run time cost. This trade-off between throughput and analysis depth is a standard problem of fuzzing-based systems [Ma21].

In the literature, a wide variety of output choices is used, such as: discrete output states [Br14; Ch16; CS15], program return values [Pe17], standard output [Pe17], simulation states [Hu21] and—for comparing compilers—the program state of the compiled programs at the end of their execution [Yal1]. Although Srivastava et al. [Sr11] employ static analysis instead of differential testing, their logging approach is worth mentioning: To find differences between Java Class Library implementations, they create a set of security-sensitive events and security checks for each function, which are compared between the implementations.

2.4 Comparison

In the comparison phase, the normalized outputs are compared to each other. If any outputs between programs differ, we found a discrepancy and thus a potential bug. Note the semantic gap here: A discrepancy between program outputs does not necessarily constitute a bug or even a vulnerability. This step can result in a lot of low-grade differences that can be regarded as noise. One example we found in our experiments was when comparing JSON parsers some parsers directly stored the substrings containing floating point numbers while others converted the strings to the data type “float”. During the casting, some information can be lost due to rounding and thus minor differences in the output can be observed. This problem is not new in the fuzzing community and different approaches exist that try to deduplicate crashes [Ma21], distinguish benign and malicious crashes [Bl20] and cluster crashes based on a root-cause analysis [Ji21]. For differential testing, Chen et al. [Ch16] reduce a discrepancy-triggering input to a simplified form.

2.5 Refinement

If the input generation is guided, the last step uses the run time information of the programs to refine the inputs. For fuzzing-based methods, the coverage information is used to guide the inputs towards unexplored paths. An interesting example is given by Petsios et al. [Pe17] who propose a coverage-guided domain-independent framework where the coverage metric named δ -diversity measures asymmetries between program execution traces.

3 Research Questions

Input generation frameworks like CSmith [Ya11], LangFuzz [HHZ12] and T-REQS [Ja21] try to accommodate generic fuzzing applications and thus work in an unguided fashion to not require the availability of source code. However, unguided methods discard valuable coverage information that could be collected to refine the input generation and find discrepancies more efficiently. Our first research question is therefore: Can unguided, structured input generation methods be improved with coverage-guided refinement? And further: Can δ -diversity [Pe17] be used as a coverage metric to improve the results?

Another important aspect of differential testing is the comparison of program outputs. Currently, programs need to generate normalized output for them to be comparable. We want to explore whether discrepancies between programs can be found by just examining their internal states. Our intuition is that, for some programs, inputs can be grouped into classes that trigger the same functionality and arrive at similar states. If statistical patterns emerge for internal states during program executions, they can be analyzed using machine learning techniques. Similar to how sentences can be summarized by a thought vector in natural language processing, the state of a program might be encoded using the execution trace. This vector representation of a program state could be used in the δ -diversity metric to learn a model of normality across programs. Deviations, i.e. anomalous behaviour, could then indicate a program discrepancy. Another similar approach is to learn to predict the program states directly from the input. If machine learning models can predict program behaviour, we can examine whether it is possible to translate between internal states of multiple programs. This could be used to search for discrepancies by predicting the state for a program, translating it to another program and then comparing the expected behaviour with the actual behaviour.

The underlying assumption in these approaches is the emergence of statistical patterns for program behaviour. Thus, our next research question is: Can statistical patterns be found in program behaviour? And lastly: Can machine learning techniques use these patterns to find discrepancies?

References

- [Ba11] Balduzzi, M.; Gimenez, C. T.; Balzarotti, D.; Kirda, E.: Automated Discovery of Parameter Pollution Vulnerabilities in Web Applications. In: NDSS. Citeseer, 2011.
- [BCS09] Barth, A.; Caballero, J.; Song, D.: Secure content sniffing for web browsers, or how to stop papers from reviewing themselves. In: 2009 30th IEEE Symposium on Security and Privacy. IEEE, 2009.
- [BI20] Blazytko, T.; Schlögel, M.; Aschermann, C.; Abbasi, A.; Frank, J.; Wörner, S.; Holz, T.: {AURORA}: Statistical Crash Analysis for Automated Root Cause Explanation. In: USENIX Security Symposium. 2020.

- [Br14] Brubaker, C.; Jana, S.; Ray, B.; Khurshid, S.; Shmatikov, V.: Using frankencerts for automated adversarial testing of certificate validation in SSL/TLS implementations. In: IEEE Symposium on Security and Privacy. IEEE, 2014.
- [Ch16] Chen, Y.; Su, T.; Sun, C.; Su, Z.; Zhao, J.: Coverage-directed differential testing of JVM implementations. In: proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation. 2016.
- [CS15] Chen, Y.; Su, Z.: Guided differential testing of certificate validation in SSL/TLS implementations. In: Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering. 2015.
- [HHZ12] Holler, C.; Herzig, K.; Zeller, A.: Fuzzing with code fragments. In: USENIX Security Symposium. 2012.
- [Hu21] Hur, J.; Song, S.; Kwon, D.; Baek, E.; Kim, J.; Lee, B.: DifuzzRTL: Differential Fuzz Testing to Find CPU Bugs. In: IEEE Symposium on Security and Privacy. IEEE, 2021.
- [Ja21] Jabiyeve, B.; Sprecher, S.; Onarlioglu, K.; Kirda, E.: T-Reqs: HTTP Request Smuggling with Differential Fuzzing. In: ACM Conference on Computer and Communications Security. 2021.
- [Ji21] Jiang, Z.; Jiang, X.; Hazimeh, A.; Tang, C.; Zhang, C.; Payer, M.: Igor: Crash Deduplication Through Root-Cause Clustering. In: Proc. of the ACM Conference on Computer and Communications Security. 2021.
- [JS12] Jana, S.; Shmatikov, V.: Abusing file processing in malware detectors for fun and profit. In: IEEE Symposium on Security and Privacy. IEEE, 2012.
- [Ma21] Manès, V. J. M.; Han, H.; Han, C.; Cha, S. K.; Egele, M.; Schwartz, E. J.; Woo, M.: The Art, Science, and Engineering of Fuzzing: A Survey. IEEE Trans. Software Eng. 47/11, 2021.
- [Mc98] McKeeman, W. M.: Differential testing for software. Digital Technical Journal 10/1, 1998.
- [Pe17] Petsios, T.; Tang, A.; Stolfo, S.; Keromytis, A. D.; Jana, S.: Nezha: Efficient domain-independent differential testing. In: 2017 IEEE Symposium on security and privacy. IEEE, 2017.
- [Sr11] Srivastava, V.; Bond, M. D.; McKinley, K. S.; Shmatikov, V.: A security policy oracle: Detecting security holes using multiple API implementations. ACM SIGPLAN Notices 46/6, 2011.
- [Ya11] Yang, X.; Chen, Y.; Eide, E.; Regehr, J.: Finding and understanding bugs in C compilers. In: Proceedings of the 32nd ACM SIGPLAN conference on Programming language design and implementation. 2011.